

Conexión a MongoDB usando un driver nativo

El driver oficial de MongoDB es la forma más directa de comunicarse con el motor de base de datos desde una aplicación. No añade capas intermedias ni modificaciones en cómo se envían y reciben los datos. El código que se ejecuta está muy cerca de la API real del servidor MongoDB.

Ventajas del driver nativo

- Se logra un acceso de máximo control:
 - Cada consulta se construye manualmente en la sintaxis propia de MongoDB
 - Las conexiones y sus parámetros se administran también de forma manual, esto permite manipular la conexión misma y las operaciones con un grado más preciso según las necesidades
- No se agregan capas de abstracción:
 - Al ser una forma de comunicación más directa, el uso de recursos de procesamiento es menor
 - Las consultas resultan más rápidas y eficientes
 - Las respuestas llegan sin ninguna transformación
- El driver es mantenido por MongoDB Inc., la empresa misma, de forma que siempre se mantiene actualizado, sin dependencias de terceros y con menor exposición a riesgos

Situaciones ideales para el driver nativo

Considerando las ventajas listadas previamente, se pueden identificar algunas situaciones donde el uso de esta forma de comunicación directa es preferible:

- Microservicios o aplicaciones sencillas, donde el agregado del ODM resulta en más trabajo que beneficio
- Aplicaciones que impliquen procesos de altos volúmenes y el rendimiento es crítico, ya que la ausencia de la capa de abstracción del ODM dará mayor rapidez y eficiencia en las operaciones
- Herramientas de administración, que realicen ajustes muy finos de configuración y rendimiento, capacidad limitada o ausente en algunos ODM
- Aplicaciones con modelo de datos cambiante. Generalmente, el uso de un ODM implica trabajar con esquemas definidos, lo que puede generar inconvenientes en aplicaciones que requieran explotar al máximo la flexibilidad

Ejemplo de uso para NodeJS

El driver nativo oficial para el ecosistema de NodeJS es [mongodb](#). Para usarlo en un proyecto de NodeJS, primero se debe inicializar el proyecto y a continuación instalar el paquete mediante el gestor *npm*. Para hacerlo corremos lo siguiente en la terminal:

```
npm init --init-type module -y
npm install mongodb
```

Luego, en el archivo de inicio de la aplicación se puede hacer uso del driver como una herramienta más incorporada al proyecto, veamos este ejemplo simple del archivo *driver.js*:

```
import { MongoClient } from 'mongodb';

const url = 'mongodb://localhost:27017/'; // String de conexión Local (sin
autenticación)
const client = new MongoClient(url);
const dbName = 'driver-test'; // Nombre de La base de datos

async function main() {
  await client.connect();
  console.log('Conexión exitosa al servidor MongoDB');
  const db = client.db(dbName); // Con el mismo cliente podría acceder a
múltiples bases de datos pasando otros valores a este método
  const collection = db.collection('documents');

  const insertResult = await collection.insertMany([ { a: 1 }, { a: 2 },
{ a: 3 } ]]); // La sintáxis es casi idéntica a La que usamos en mongosh
  console.log('Documentos insertados =>', insertResult);

  return 'Completado.';
}

try {
  const result = await main();
  console.log(result);
} catch (error) {
  console.error('Error en la conexión con MongoDB:', error);
} finally {
  await client.close();
}
```

Para realizar la prueba:

```
node driver.js
```

